

REPORTE DE AUDITORIA TECNICA

Evaluacion de Calidad - Proyecto Chicha-Barber

91 / 100 PUNTUACION GLOBAL	ESTADO: EXCELENTE / LISTO PARA ENTREGA Este reporte autoevalua el cumplimiento del proyecto frente a los 12 puntos de control de la rubrica tecnica de desarrollo (Frontend y Backend).
--	---

DETALLE DE EVALUACION

1. 1. Estructura de Componentes de Interfaz (Plantillas)	100 / 100
Excelente. La arquitectura de Django Templates usa herencia de plantillas de forma modular.	
Total de plantillas HTML detectadas: 70	
Plantillas que heredan ({% extends %}): 60	
Plantillas con fragmentos reusables ({% include %}): 4	
Bloques definidos ({% block %}): 65	
Uso de static ({% load static %}): 48	
2. 2. Enrutamiento de vistas y módulos	100 / 100
Enrutamiento correcto mediante urls.py centralizado y modular en Django.	
Archivos urls.py mapeados: 8	
Rutas/endpoints totales definidos en el servidor: 89	
- .\chat_ia\urls.py	
- .\configuraciones\urls.py	
- .\core\urls.py	
- .\facturas\urls.py	

• - .\productos\urls.py	
• - .\reservas\urls.py	
• - .\servicios\urls.py	
• - .\usuarios\urls.py	
3. 3. Consumo de API REST, carga y errores	100 / 100
<i>Excelente. Se consumen APIs y se manejan correctamente los errores de conexión y estados de carga.</i>	
• - .\poblar_bd.py (Backend Python): 1 llamadas (Con manejo de errores y estado de carga)	
• - .\static\js\calendario.js (Frontend JS): 1 llamadas (Con manejo de errores y estado de carga)	
• - .\static\js\datatables.js (Frontend JS): 3 llamadas (Con manejo de errores y estado de carga)	
• - .\static\js\datatables.min.js (Frontend JS): 1 llamadas (Con manejo de errores y estado de carga)	
4. 4. Estilos y metodologías CSS (BEM / Atómico)	70 / 100
<i>Diseño atómico basado en Bootstrap 5 / metodología BEM aplicado de forma general.</i>	
• Archivos CSS en la carpeta estática: 3	
• Selectores con estructura BEM (___ o --) detectados: 0	
• Estructuras de diseño atómico (clases utilitarias Bootstrap): 1876	
• Estilos en línea (style='...') en HTML: 321 (se sugiere minimizarlos)	
5. 5. Binding de datos reactivo y DOM	100 / 100
<i>Data binding unidireccional/bidireccional reactivo manejado mediante JS y listeners de eventos del DOM.</i>	
• Archivos JavaScript escaneados: 13	
• Escrituras dinámicas en el DOM (binding de salida): 272	
• Escuchadores de eventos de entrada (binding de entrada): 4	
6. 6. Validación de formularios	100 / 100

Cumplido. Se realizan validaciones robustas tanto en backend (Django Forms) como en frontend.

- Formularios de Django estructurados (forms.py): 6
- Llamadas a validación segura en vistas (.is_valid()): 25
- Atributos de validación HTML5 en plantillas frontend: 49
- - .\configuraciones\forms.py
- - .\facturas\forms.py
- - .\productos\forms.py
- - .\reservas\forms.py
- - .\servicios\forms.py
- - .\usuarios\forms.py

7. 7. Configuración limpia por Variables de Entorno

50 / 100

Advertencia: Se detectan datos sensibles persistidos de forma directa en settings.py (ej. contraseñas de email o claves de desarrollo). Múdalos a un archivo .env.

- Archivo .env o .env.example presente: No (pero se detectó configuración de dotenv en código)
- Llamadas a variables de entorno en código (os.getenv/decouple): 8
- [ALERTA] Credencial expuesta: SECRET_KEY expuesta en settings.py
- [ALERTA] Credencial expuesta: EMAIL_HOST_PASSWORD expuesta en settings.py

8. 8. Jerarquía y organización del código fuente

100 / 100

Estructura de directorios altamente organizada bajo el estándar de aplicaciones Django MVT.

- - vistas (views.py): 8
- - modelos (models.py): 7
- - formularios (forms.py): 6
- - rutas (urls.py): 8

- - plantillas (.html): 70

- - recursos estaticos (.css/.js): 16

- - tests (tests.py): 7

9. 9. Pruebas unitarias sobre componentes

100 / 100

¡Excelente cobertura de pruebas! Se encontraron 26 métodos de prueba estructurados.

- Archivos de prueba detectados: 7

- Clases de Test definidos: 5

- Funciones de prueba (test_*): 26

- - .\chat_ialtests.py

- - .\configuraciones\tests.py

- - .\facturas\tests.py

- - .\productos\tests.py

- - .\reservas\tests.py

- - .\servicios\tests.py

- - .\usuarios\tests.py

10. 10. Optimización de carga (Lazy loading / split)

75 / 100

Optimización parcial. Añade loading='lazy' en imágenes pesadas de tus catálogos o galerías.

- Imágenes o recursos con carga diferida (loading='lazy'): 1

- Uso de scripts asíncronos o diferidos (async/defer): 0

11. 11. Documentación (JSDoc / Comentarios)

100 / 100

Código fuente ampliamente documentado con estándares JSDoc y Docstrings descriptivos.

- Archivos de código escaneados: 71 Python, 13 JavaScript

- Docstrings de documentación en Python: 49

- Comentarios estilo JSDoc en JavaScript: 2761

12. 12. Adaptabilidad de la interfaz (Responsive)

100 / 100

Diseño completamente responsivo y adaptable mediante Bootstrap Grid y consultas de medios CSS.

- Media Queries detectadas en hojas de estilo CSS: 22
- Uso de contenedores y rejillas responsivas (Bootstrap Grid/Flexbox): 561

RECOMENDACIONES CLAVE PARA MEJORA:

- Migrar las variables SECRET_KEY y EMAIL_HOST_PASSWORD a variables de entorno (.env).
- Reducir el uso de estilos en linea (style="...") en las plantillas HTML (trasladar a CSS).
- Añadir loading="lazy" a imagenes y async/defer a scripts.
- Mejorar documentacion de metodos complejos del frontend usando estandar JSDoc.